

Introduction — Chapter 1

2023年9月

叶冠华

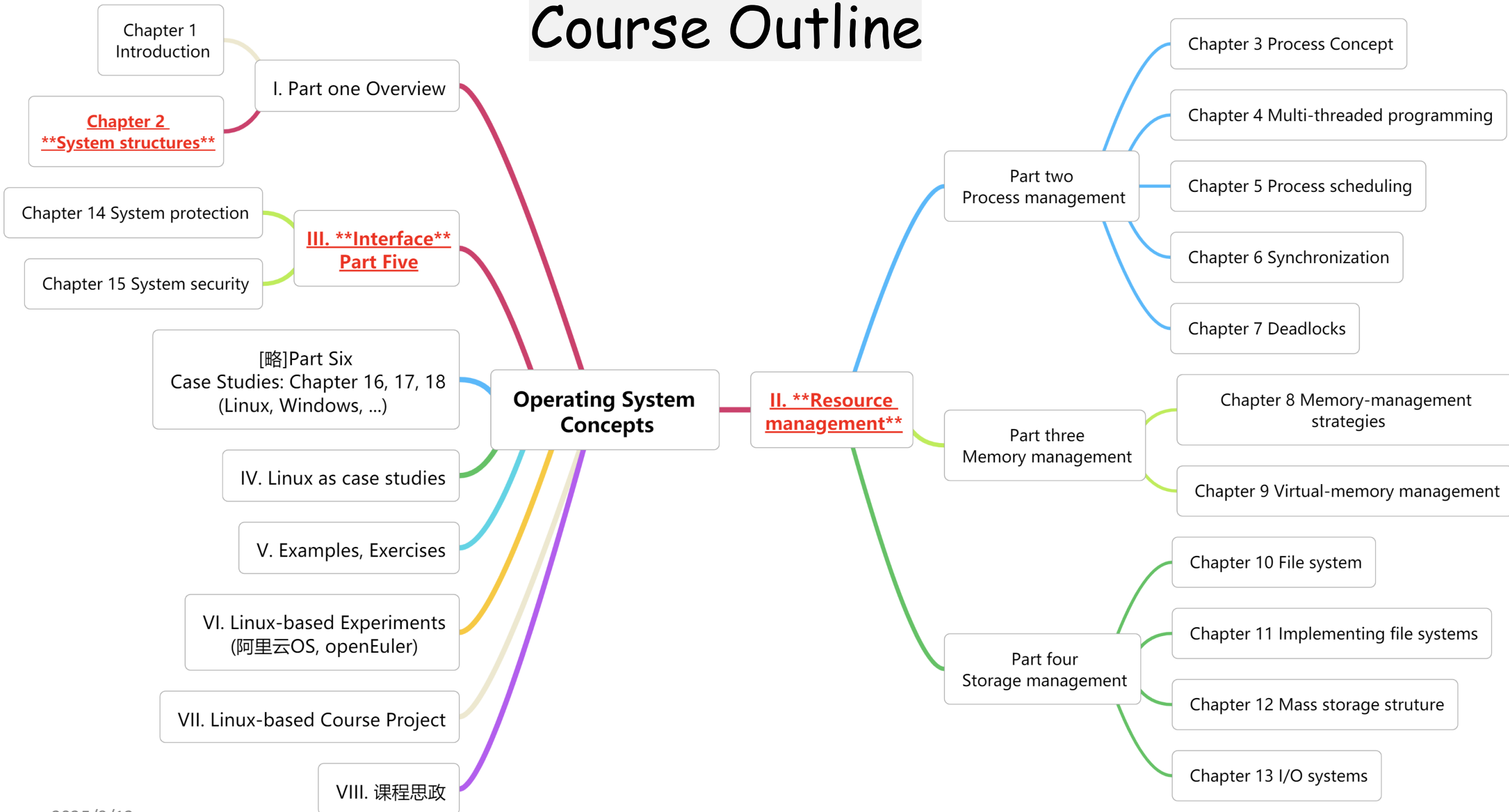
g.ye@bupt.edu.cn

School of Computer Science (National Pilot Software Engineering School)



北京邮电大学

Course Outline



Course Assessment and Grading

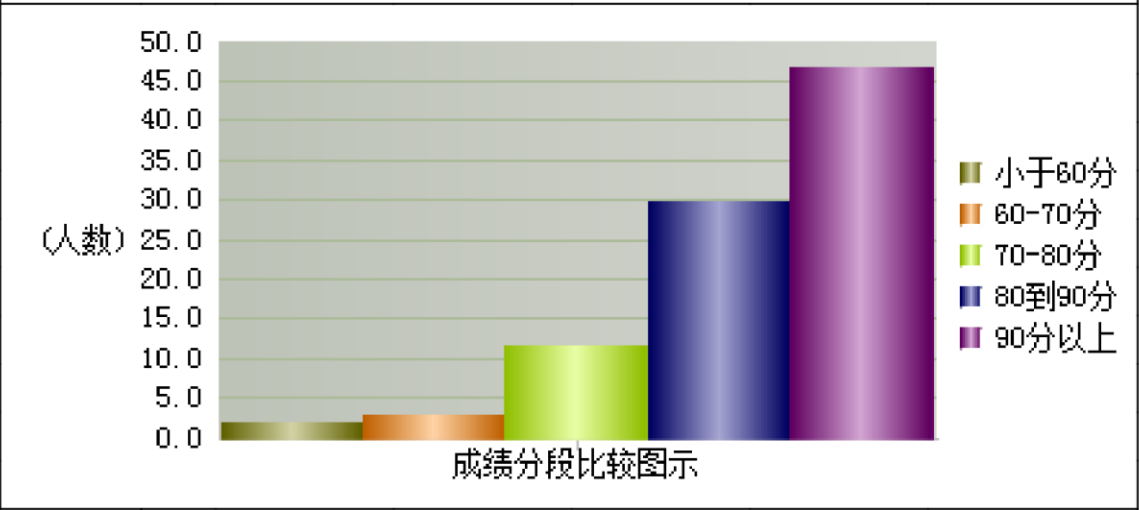
■ Mid-term test, exercises, experiments, final examination

■ Grading: 20% + 10% + 10% + 60%

■ 考试/测验, 作业

■ 英文试卷/题目, 中文答题

成绩构成	实验成绩:10%; 平时成绩:10%; 期中成绩:20%; 期末成绩:60%				
考试方式	考试	成绩类型	百分制		
分数段	0-59.99	60-69.99	70-79.99	80-89.99	90-100
人数	2	3	12	30	47
百分比	2.1%	3.2%	12.8%	31.9%	50%
上课人数	95	考试人数	94		
平均分	85.85	最高分	98	最低分	52
不参与计算	免修	缓考	休学	缺考	违纪
	0	1	0	0	0



Chapter 1 Introduction

I. 1.1 What operating systems do (definition, functions/services)

resource allocation + control program

interfaces for user-machine interaction

II. Basic organization of computer systems (hardware)

1.2 Computer-system organization: startup/booting, interrupt, I/O operation&structure, storage hierarchy

1.3 Computer-system architecture: multiprocessing, dual/multi-core processor, cluster systems

III. 1.4 Operating-system structures (approaches for supporting concurrent program execution)

multiprogramming

timesharing

IV, 1.5 Operating-system operations (CPU oriented)

privilege instructions
&dual-mode operation;
kernel mode&user mode
kernel space/user space

IX. 课程思政：国产OS/系统软件, 信创

V. Grand tour of OS major components /functions 【略】

1.6 Process management,
1.7 Memory management,
1.8 Storage management,
1.9 Security and protection,
1.10 Kernel data structures

VI. 1.11 Computing environments (hardware,software, architecture, operating schemes)

OS in:
traditional, mobile,
distributed, client-server,
peer-to-peer,
virtualization,
cloud computing,
fog/edge computing,
real-time and embedded

VII. 1.12 Open-source operating systems

VIII. Linux as case studies-Introduction

X. Appendix

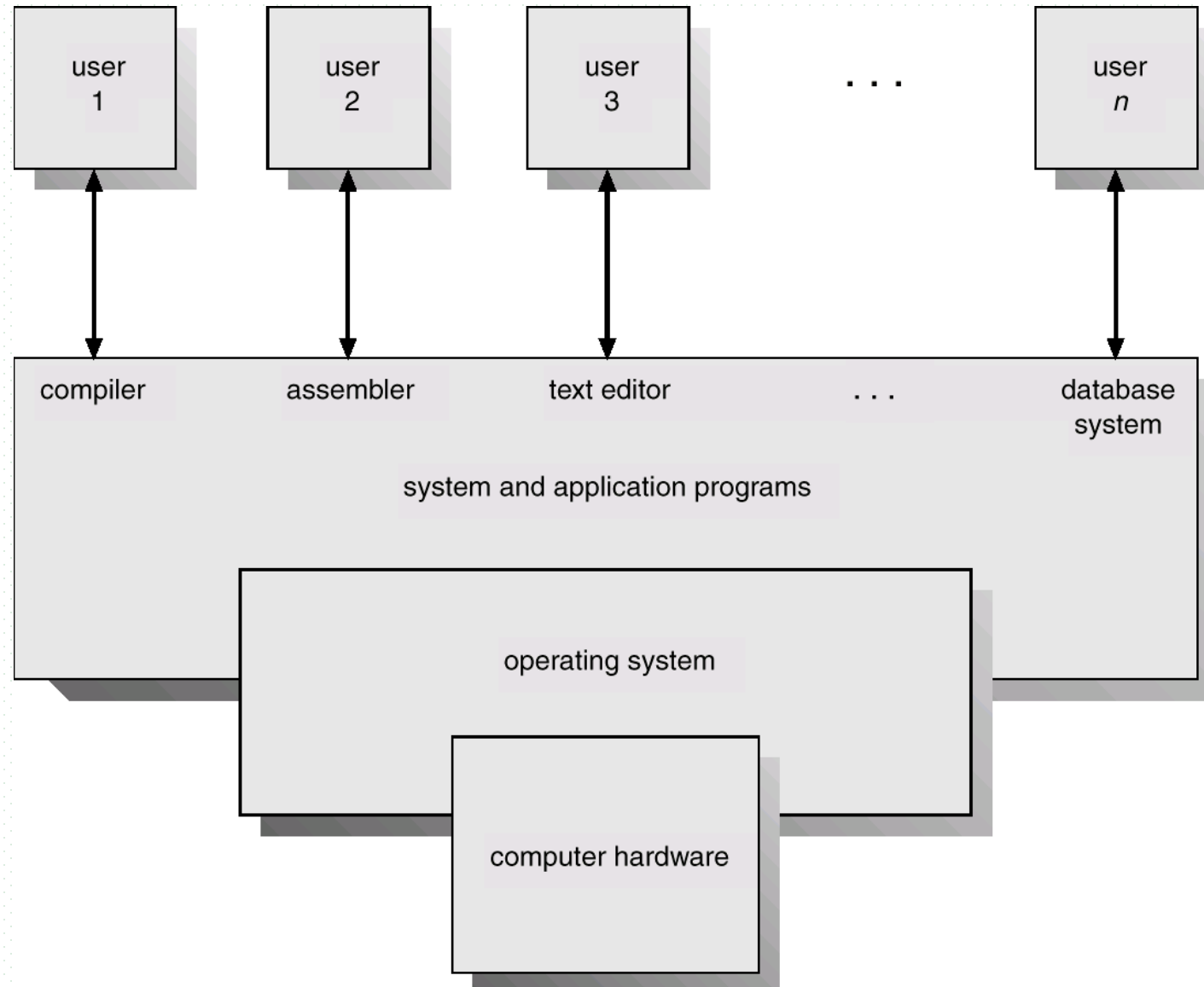
Appendix 1-1 操作系统生态

Appendix 1-2 信息技术应用创新 (信创)

Appendix 1-3 OS研究热点和相关顶会

1.1 What Operating Systems Do

- What is a operating system
 - ▣ a program that acts as an intermediary between a user of a computer and the computer hardware
 - ▣ user
 - persons, programs, ...
- Operating system goals
 - ▣ execute user programs and make solving user problems easier
 - ▣ make the computer system **convenient** to use
 - ▣ use the computer hardware in an **efficient** manner



Computer System Structure

- Computer system can be divided into four components
 - ▣ hardware – provides basic computing resources
 - CPU, memory, I/O devices
 - ▣ operating system
 - controls and coordinates use of hardware among various applications and users
 - ▣ application programs – define the ways in which the system resources are used to solve the computing problems of the users
 - word processors, compilers, web browsers, database systems, video games
 - ▣ users
 - people, machines, other computers

What Operating Systems Do

- Depends on the points of view, that is, **user** view and **system** view
- User view
 - convenient interface for users to use computers, **ease of use**
 - providing **good performance**
- Shared computer such as **mainframe (大型主机)** or **minicomputer (小型机)** must keep all users happy
- Users of dedicate systems such as **workstations (工作站)** have dedicated resources but frequently use shared resources from **servers (服务器)**
- Handheld computers(手持计算机) are resource poor, optimized for usability and battery life
- Some computers have little or no user interface, such as **embedded (嵌入式) computers** in devices and automobiles
- System view
 - **resource allocation for programs running on computers**
 - efficient management of hardware (CPU, memory, I/O devices, storage) and software (program, data) resources in computer
 - **maximization of utilization (利用率)**

Operating System Definition

- No universally accepted definition, can be described from following aspects
 - ▣ OS is a **resource allocator** (分配器)
 - manages all hardware (e.g. CPU) and software (e.g. file) resources in computers
 - decides between conflicting requests for efficient and fair resource use
 - ▣ OS is a **control program**
 - controls execution of programs to prevent errors and improper use of the computer
 - ▣ OS provides **interface** between computer hardware and other system and application programs
 - GUI, mouse, keyboard, ...
- As system software, OS = **kernel** (内核) + system programs
 - ▣ "the one program running at all times on the computer" is the **kernel**
 - ▣ everything else is either
 - a system program (ships with the operating system) , or
 - an application program

Operating System Definition

■ Summary

- ❑ OS is a type of system software, residing between computer hardware and the other software or users in system
- ❑ OS provides the **interfaces** (接口) for the users and application programs to use computer software and hardware resources **conveniently**
- ❑ it also **efficiently** manages and controls hardware resources, such as CPU, memory, and I/O devices, to maximize the utilization of these resources; also manage information (e.g. programs, data) resources

Computer System Organization

■ I/O operation

- ❑ I/O devices and the CPU can execute concurrently (所以进程在I/O时退出执行状态可以提高系统整体效率)
- ❑ each device controller (某种硬件) is in charge of a particular device type
- ❑ each device controller has a local buffer
- ❑ CPU moves data from/to main memory to/from local buffers (内存 $\leftrightarrow$ Buffer) (需要CPU)
- ❑ I/O is from the device to local buffer of controller (Buffer $\leftrightarrow$ 设备) (无需CPU)
- ❑ device controller informs CPU that it has finished its operation by causing an **interrupt** (中断)

Interrupt

■ Interrupts in computers

- ❑ when some events occur, the programs currently running on the CPU are interrupted, CPU control is transferred to the *interrupt service routine (ISR)* to handle the events
- ❑ interrupt mechanism: interrupt enabling/disabling, interrupt vector, interrupt priority

■ Fig. 1.3 Interrupt time line for a single process doing input, e.g. disk reading

- ❑ (1) user program runs on CPU, and I/O device is idle
- ❑ (2) user program prepares memory buffer, send a I/O request to I/O device by I/O API (e.g. **read** file) and drivers, ready to doing input
- ❑ (3) I/O device responds to the request, begins to transfer data, I/O device and the CPU execute in parallel
- ❑ (4) after it has finished its I/O operations, I/O device informs CPU by causing an I/O **interrupt**, and CPU switches to interrupt service routine (i.e. drivers) to fetch the data from disk; I/O device transits into the idle state
- ❑ (5) after I/O interrupt processing is over, CPU returns to user program

Interrupt

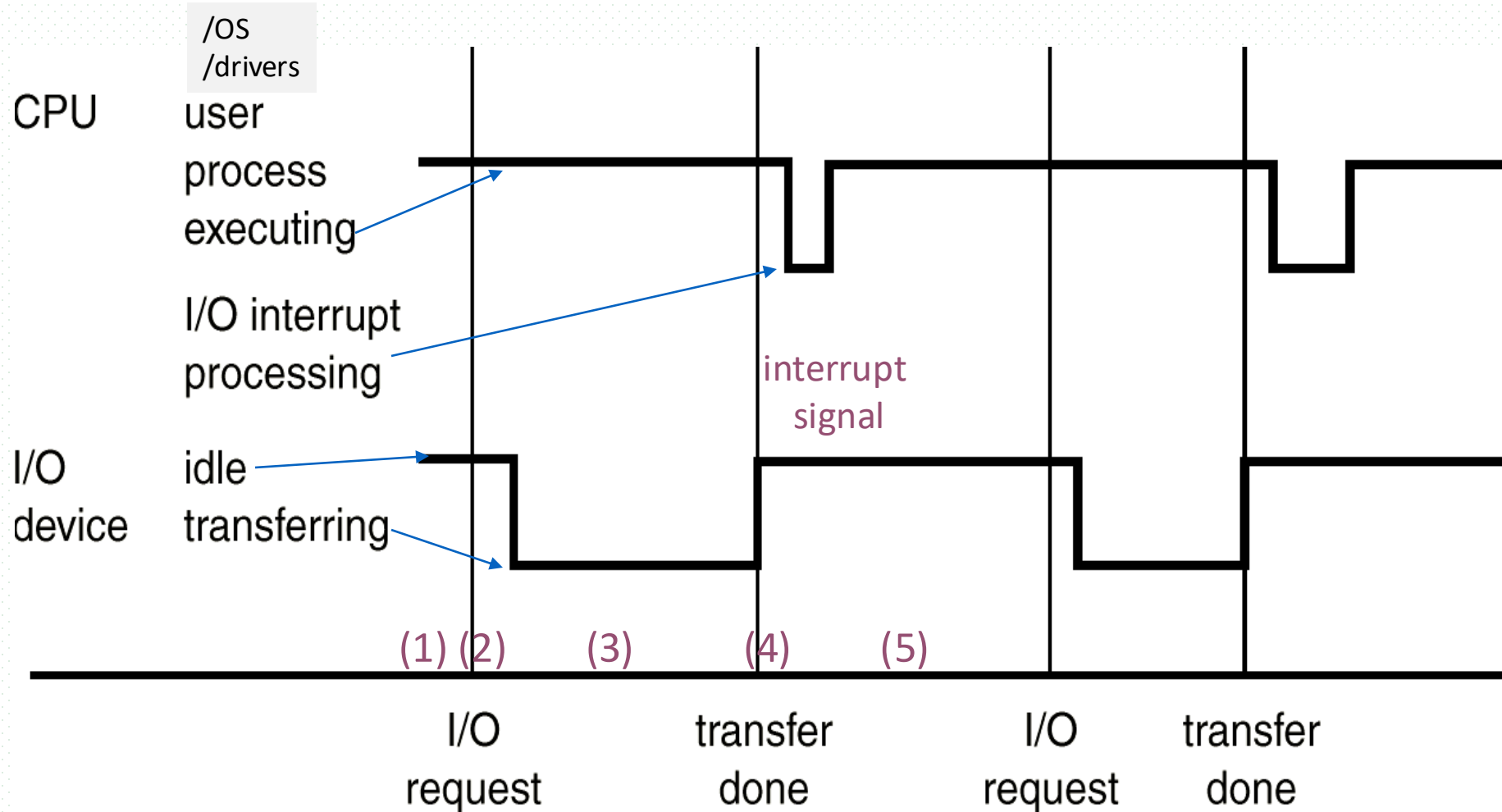


Fig.1.3 Interrupt time line for a *single* process doing output
e.g. input data from disk

Common Functions of Interrupts

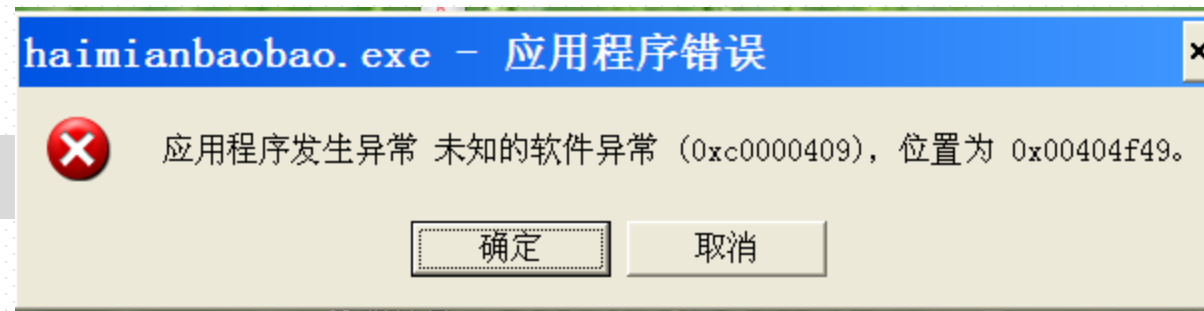
- An operating system is interrupt driven
- Interrupt transfers control to the *ISR* generally, through the **interrupt vector(中断向量)**, which contains the addresses of all the service routines
 - ▣ interrupt architecture must save the address of the interrupted instruction
 - ▣ OS preserves the state of the CPU by storing registers and the program counter
- Determines which type of interrupt has occurred:
 - ▣ **polling (轮询)**
 - ▣ **vectored** interrupt system
- Separate segments of code determine what action should be taken for each type of interrupt

Interrupt (in General)

- Three types of interrupts
- Hard interrupt
 - ▣ caused by hardware interrupt signals, triggered by external events
- System call/monitor call (系统调用)
 - ▣ an interrupt triggered by software
 - ▣ from user request program that a OS service be performed, e.g. **system calls** such as *read*
- Trap (陷阱) or exception (异常)
 - ▣ software-generated interrupt caused e by an error



e.g. 2+3/0



Storage Definitions and Notation Review

The basic unit of computer storage is the **bit**. A bit can contain one of two values, 0 and 1. All other storage in a computer is based on collections of bits. Given enough bits, it is amazing how many things a computer can represent: numbers, letters, images, movies, sounds, documents, and programs, to name a few. A **byte** is 8 bits, and on most computers it is the smallest convenient chunk of storage. For example, most computers don't have an instruction to move a bit but do have one to move a byte. A less common term is **word**, which is a given computer architecture's native unit of data. A word is made up of one or more bytes. For example, a computer that has 64-bit registers and 64-bit memory addressing typically has 64-bit (8-byte) words. A computer executes many operations in its native word size rather than a byte at a time.

Computer storage, along with most computer throughput, is generally measured and manipulated in bytes and collections of bytes.

A **kilobyte**, or **KB**, is $1,024$ bytes

a **megabyte**, or **MB**, is $1,024^2$ bytes

a **gigabyte**, or **GB**, is $1,024^3$ bytes

a **terabyte**, or **TB**, is $1,024^4$ bytes

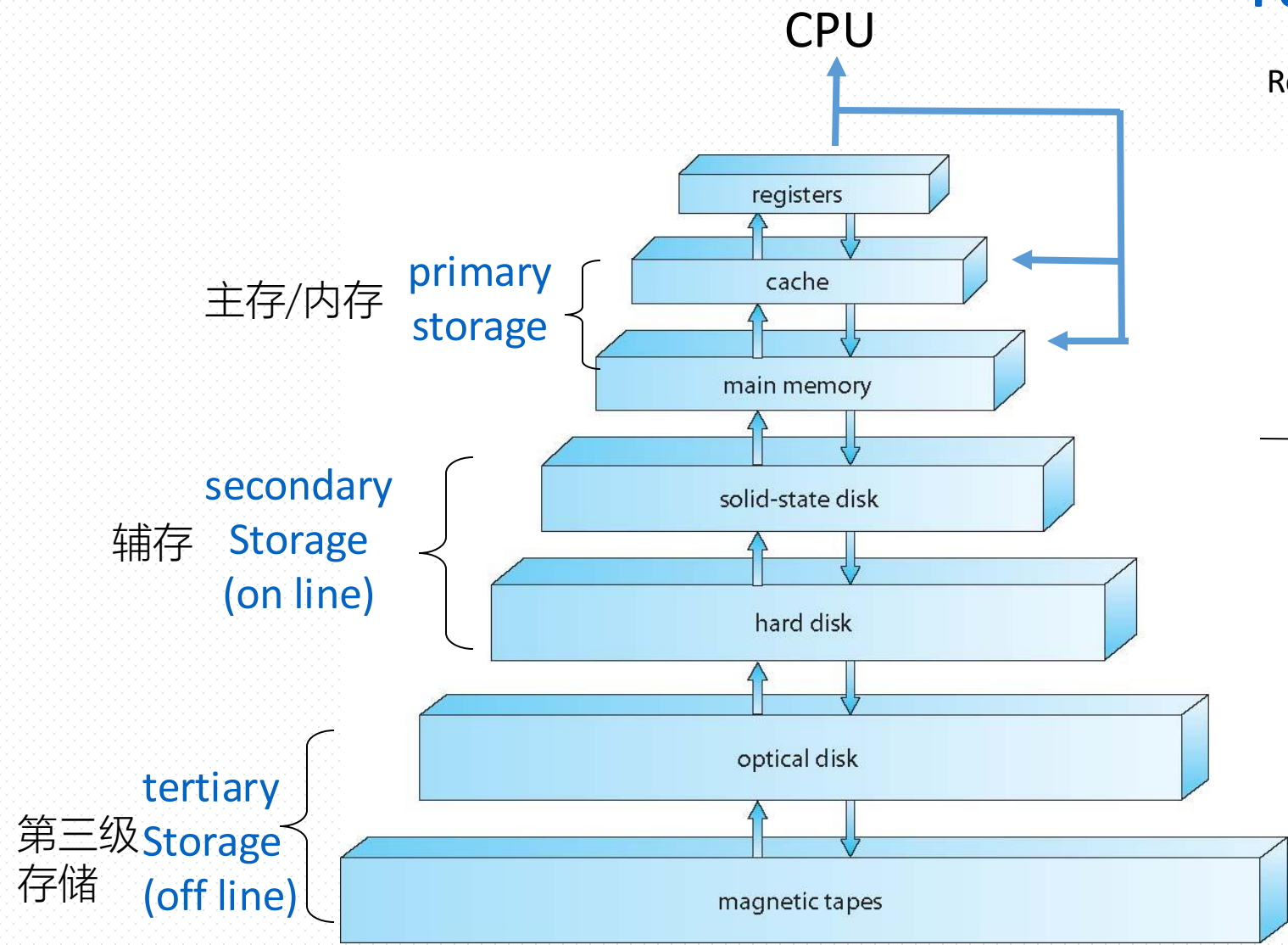
a **petabyte**, or **PB**, is $1,024^5$ bytes

Computer manufacturers often round off these numbers and say that a megabyte is 1 million bytes and a gigabyte is 1 billion bytes. Networking measurements are an exception to this general rule; they are given in bits (because networks move data a bit at a time).

Storage Structure

- Storage systems organized in hierarchy
 - ▣ speed
 - ▣ cost(成本)
 - ▣ volatility (易失性)
 - ▣ access mode(访问模式)

Storage-Device Hierarchy



Performance Indicators

Reliability	Speed	Capacity	Cost /per unit of memory	Access Mode
volatile	fast	small	high	byte-based
nonvolatile	slow	large	low	block-based

Storage Structure

- Main memory – only large storage media that the CPU can access directly
 - **random access** (随机访问)
 - typically **volatile**-易失型
- Secondary storage
 - extension of main memory that provides large **nonvolatile** storage capacity
- Hard disks – rigid metal or glass platters covered with magnetic recording material
 - disk surface is logically divided into **tracks** (磁道), which are subdivided into **sectors** (扇区)
 - the **disk controller** determines the logical interaction between the device and the computer
- **Solid-state disks(SSD, 固态硬盘)** – faster than hard disks, nonvolatile
 - various technologies, e.g.
 - becoming more popular

Caching

■ Caching (缓存)

- ▣ copying information in use from slower to faster storage temporarily, e.g.
- Important principle, performed at many levels in a computer (in hardware, operating system, software)
 - ▣ L1/L2/L3 cache in CPU
 - ▣ main memory can be viewed as a cache for secondary storage, in DBS
- Faster storage (cache) checked first to determine if information is there
 - ▣ if it is, information used directly from the cache (fast)
 - ▣ if not, data copied to cache and used there
- Cache smaller than storage being cached
 - ▣ cache management important design problem
 - ▣ cache size and replacement policy

■ Buffer (缓冲区)

- ▣ used in I/O

多级缓存cache

■ 一级 Cache

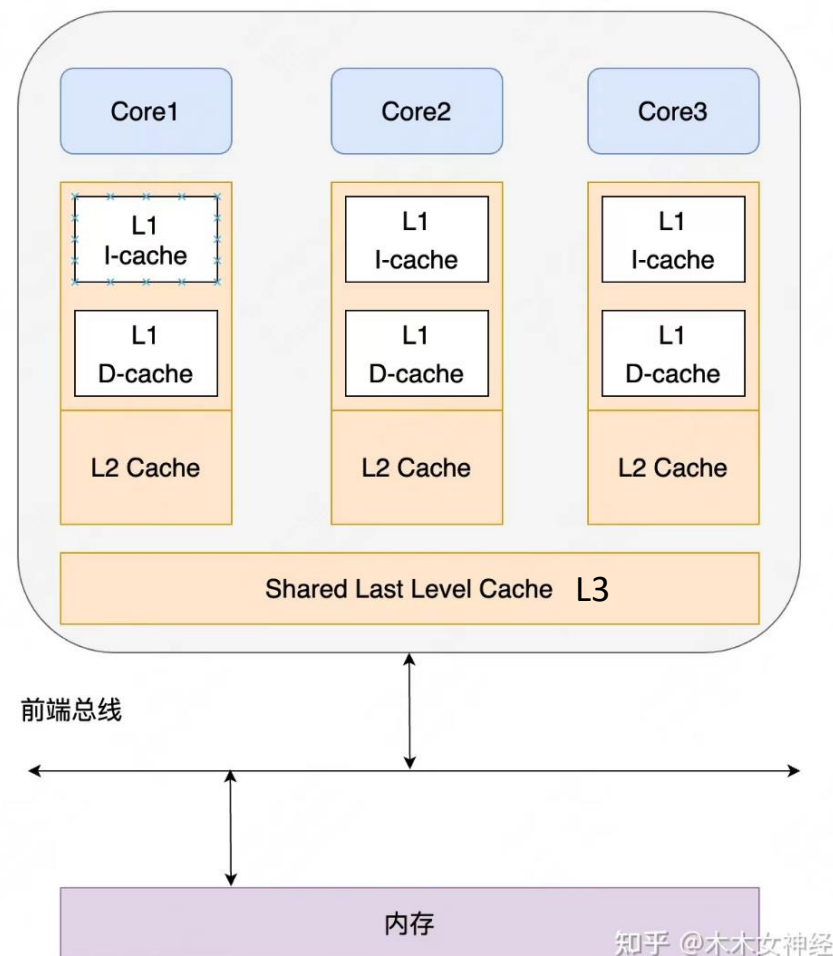
- ❑ 分为数据 Cache 和指令 Cache
- ❑ 速度最快，数据访问速度3~5个指令周期
- ❑ 成本高，容量小只有几十 KB
- ❑ 每个CPU核拥有自己的一级 Cache

■ 二级 Cache

- ❑ 数据和指令无差别地存放在一起
- ❑ 速度比一级 Cache 慢，数据访问速度十几个指令周期
- ❑ 容量较大，几百 KB 到几 MB 不等
- ❑ 每个CPU核拥有自己的二级 Cache

■ 三级Cache

- ❑ 速度更慢，数据访问速度几十个指令周期
- ❑ 容量大，几 MB 到几十 MB
- ❑ 为多核CPU内部所有核所共有



I/O Structure

- A computer system consists of CPUs and multiple **device controllers**(设备控制器) that are connected through a common bus, and each device controller is in charge of a specific type of I/O **device**
 - ▣ depending on the controller, more than one device may be attached
 - ▣ e.g. seven or more disk devices can be attached to the **small computer-systems interface (SCSI)** controller
- A device controller maintains some local buffer storage and a set of special-purpose registers, and is responsible for moving the data between the devices that it controls and its local buffer storage
 - ▣ status register, control register
 - ▣ local buffer storage
 - input register, output register
- These registers and local buffer storage are addressable for CPU, known as **I/O ports**(I/O 端口)
- I/O hardware: device, device controller, I/O ports

I/O Structure

- OS has a **device driver (设备驱动程序)** for each device controller to manage I/O
 - ▣ provides uniform interface between controller and kernel
- CPU executes the driver to transfer data between devices and main memory, e.g. “read a character from the keyboard”
 - ▣ the device driver loads the appropriate registers (e.g. control register) within the device controller—发出I/O控制命令
 - ▣ the device controller, in turn, examines the contents of these registers to determine what action to take (such as “read a character from the keyboard”)
 - ▣ the controller starts the transfer of data from the device to its local buffer:
 - keyboard → input register
 - ▣ once the transfer of data is complete, the device controller informs the device driver via an interrupt that it has finished its operation
 - ▣ the device driver fetch the data or a pointer to the data to main memory
 - input register → main memory
 - ▣ the driver reads status register to watch device operating status
 - e.g. the device is idle, I/O operation is completed

I/O Structure

- Direct memory access (**DMA**, 直接内存访问) is used for bulk data movement such as disk I/O

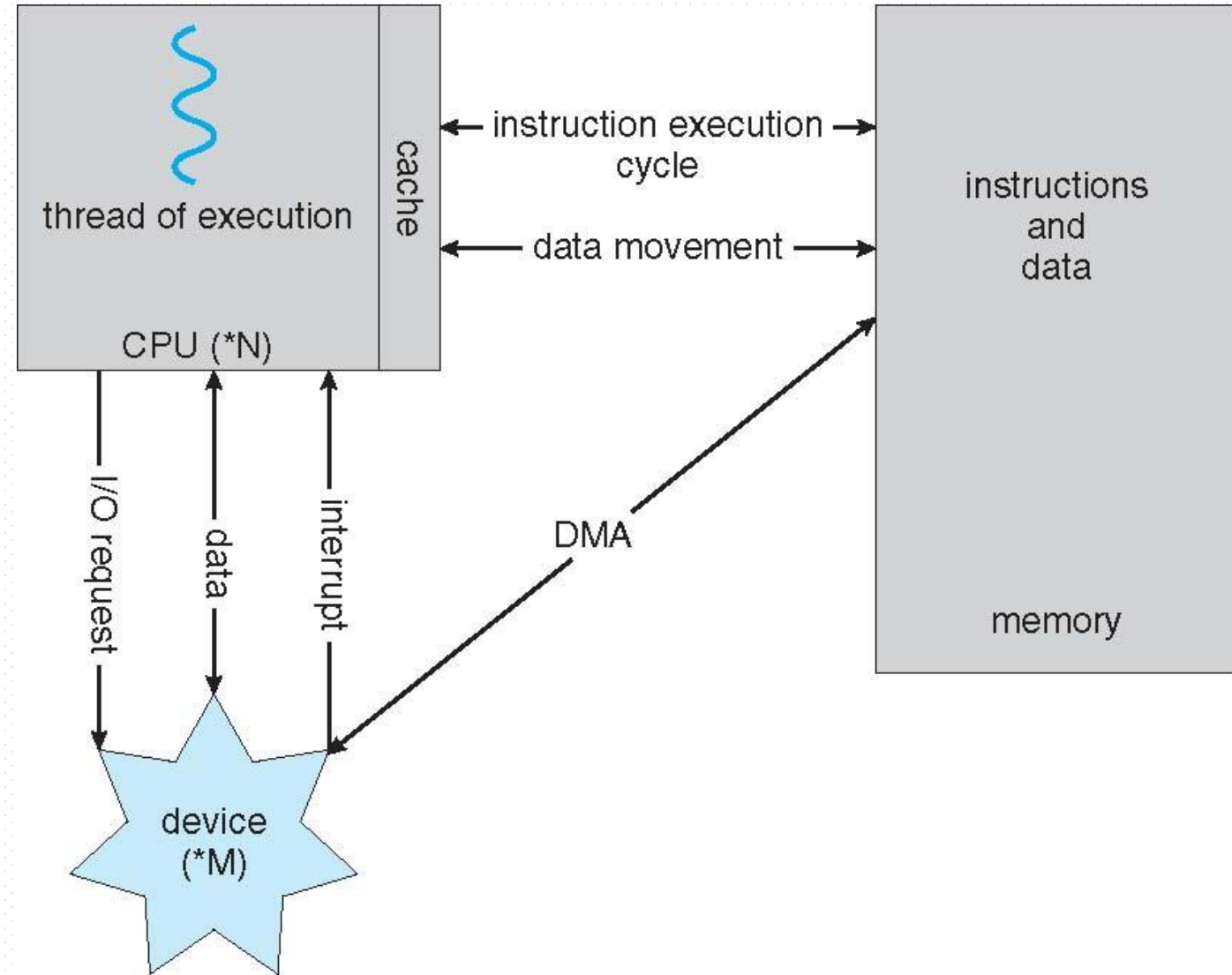
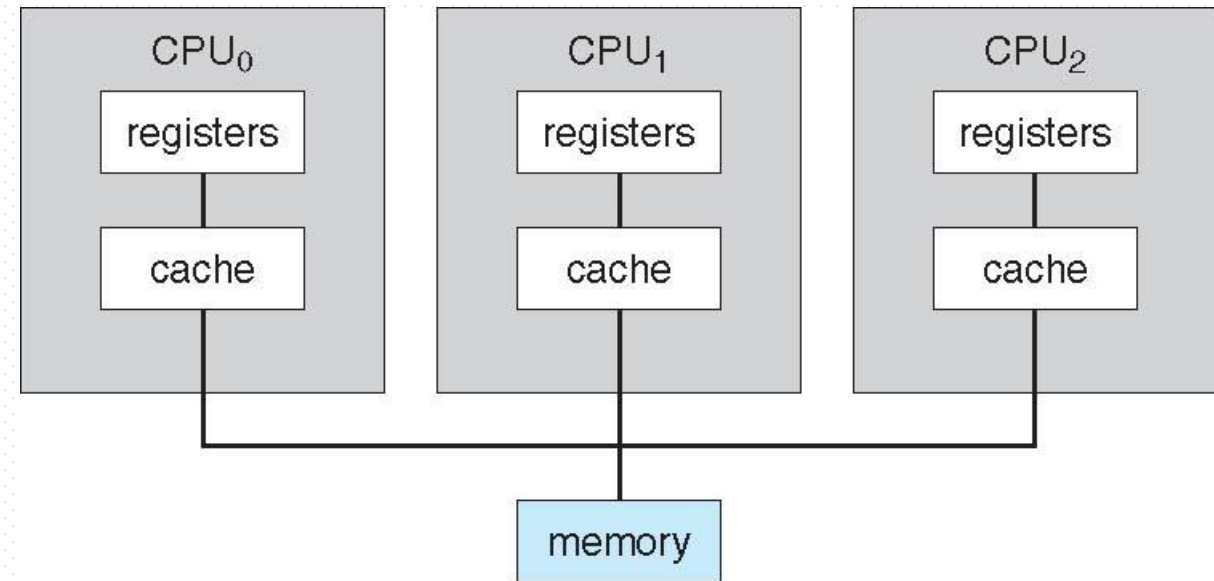


Figure 1.5 How a Modern Computer Works

1.3 Computer-System Architecture

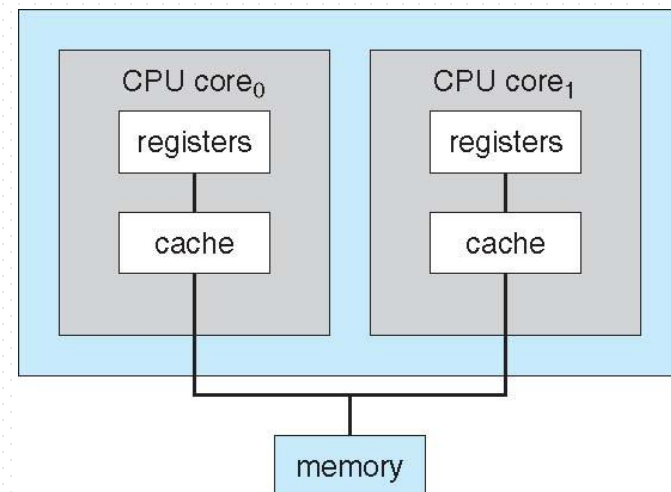
- Most systems use a single general-purpose processor, however maybe **multi-core (多核)** in a processor
- Multiprocessors (多处理器) systems growing in use and importance
 - ▣ also known as parallel systems, tightly-coupled (紧耦合) systems
 - ▣ advantages include:
 - increased throughput (吞吐量)
 - economy of scale
 - increased reliability(可靠性)
 - graceful degradation (性能降阶) or fault tolerance (容错)
 - ▣ two types
 - symmetric (对称) multiprocessing – each processor performs all tasks
 - asymmetric multiprocessing – each processor is assigned a specific task



Dual-Core/Multi-Core Design

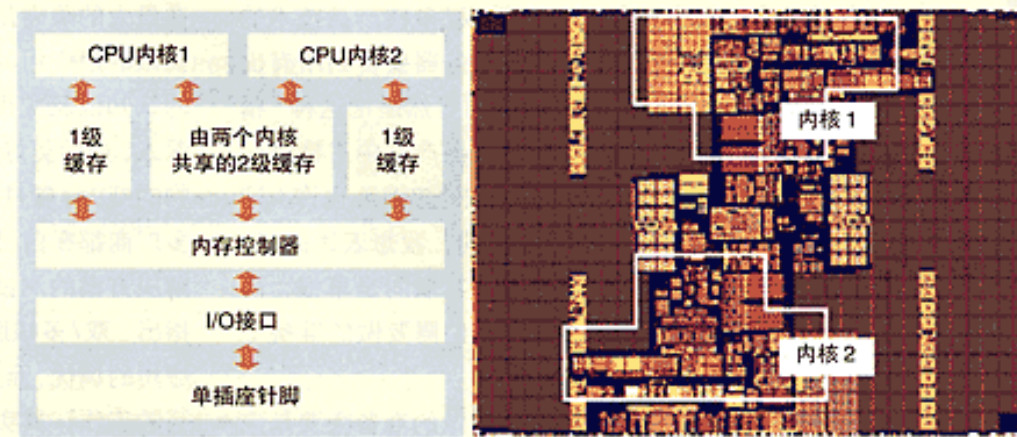
- 双核/多核/众核CPU
- Multi-chip and **multicore**
- 神威太湖之光高性能计算机
 - 申威26010众核处理器
 - 25平方厘米，260个运算核心
 - 每秒3万+亿次计算能力

- 飞腾64核CPU芯片
 - 64核通用CPU，ARM架构
 - 浮点运算的峰值速度5120亿次/秒，
 - 用于天河超级计算机管理结点



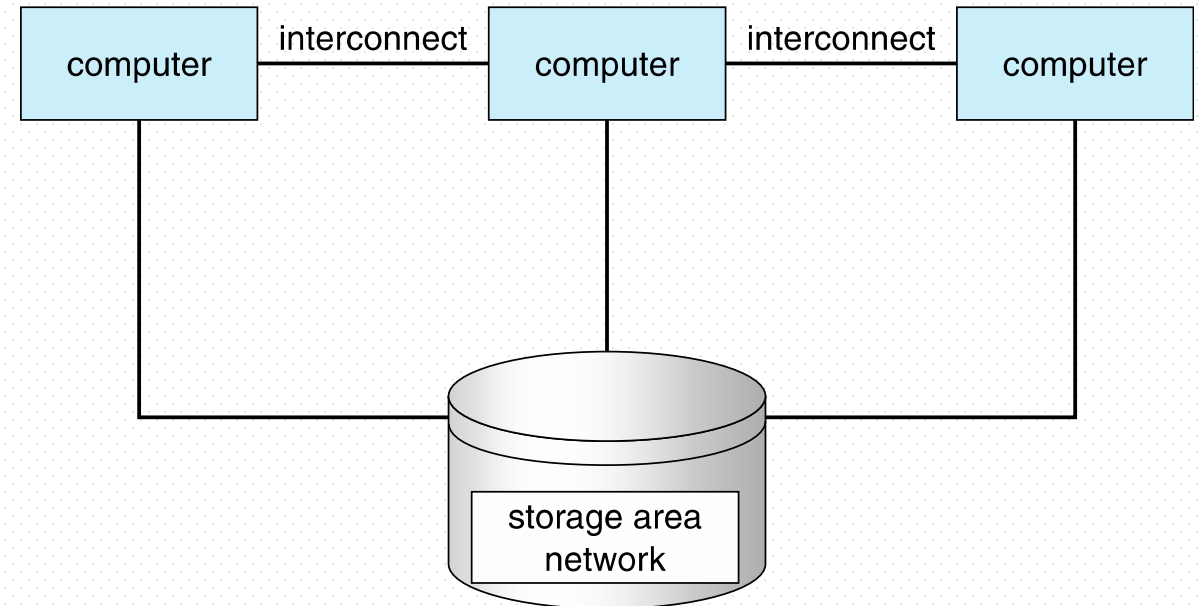
多内核架构

目前的通用多内核处理器设计都是基于对称多处理 (SMP) 技术的。SMP 是一种并行架构，在这种架构中，所有的处理器运行一个操作系统副本，共享内存和一台计算机的其他资源，并且平等地访问内存、I/O 及外部中断。大多数 SMP 架构是由下图演化而来的：



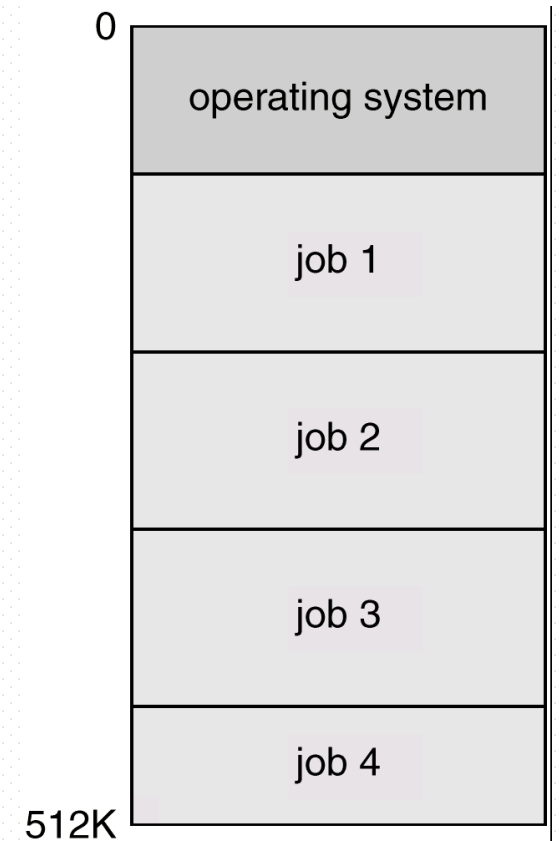
Clustered Systems (集群系统)

- Like multiprocessor systems, but multiple systems working together
 - ▣ usually sharing storage via a **storage-area network (SAN)**
 - ▣ provides a **high-availability (高可用性)** service which survives failures
 - **asymmetric clustering** has one machine in hot-standby mode
 - **symmetric clustering** has multiple nodes running applications, monitoring each other
 - ▣ some have **distributed lock manager (DLM)** to avoid conflicting operations
 - ▣ some clusters are for **high-performance computing (HPC)**
 - applications must be written to use **parallelization**
- 曙光天潮集群系统 in BUPT
 - ▣ 5000亿次浮点运算/秒
 - ▣ 40个计算节点



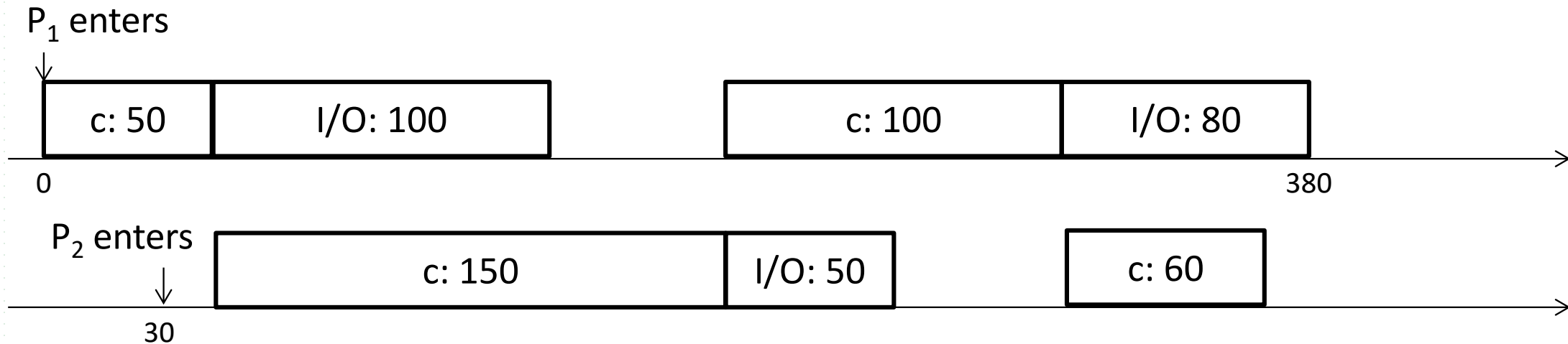
1.4 Operating System Structure

- OS mechanisms to support multiple programs concurrently (并发) execution in system
 - ▣ multiprogramming, time-sharing
- Multiprogramming/Batch system (多道程序设计/批处理系统) needed for efficiency
 - ▣ single user cannot keep CPU and I/O devices busy at all times
 - ▣ multiprogramming organizes jobs (code and data) so CPU always has one to execute
 - ▣ a subset of total jobs in system is kept in memory
 - ▣ one job selected and run via [job scheduling](#)
 - refer to long-term, medium-term, short-term scheduling in chapter 5
 - ▣ when it has to wait (for I/O for example), OS switches to another job



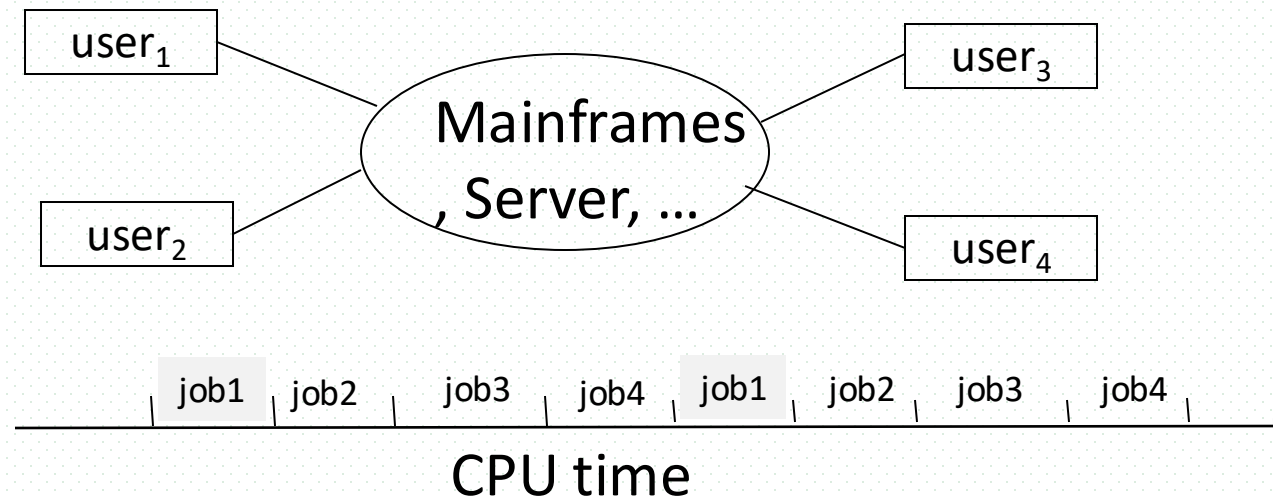
Example: concurrent scheduling in batch system by multiprogramming

- In a multiprogramming batch system, there are two jobs/programs/processes P_1 and P_2 concurrently running on a single CPU. P_2 enters into the system 30ms later than P_1 , and their executing traces, i.e. alternating sequences of CPU bursts and I/O bursts, are as follows
 - ▣ P_2 : computing, 150ms $\rightarrow$ I/O operation, 50ms $\rightarrow$ computing, 60ms
 - ▣ P_1 : computing, 50ms $\rightarrow$ I/O operation, 100ms $\rightarrow$ computing, 100ms $\rightarrow$ I/O operation, 80ms

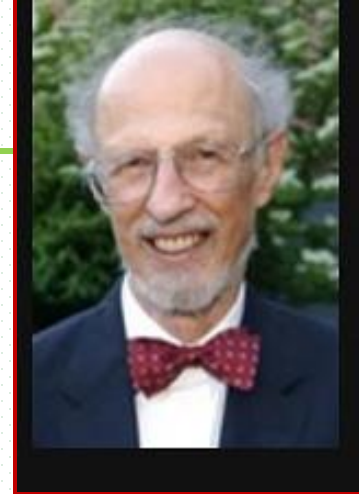


Operating System Structure

- **Timesharing (分时)** is logical extension in which CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive (交互式)** computing
 - ▣ CPU times are divided into timeslot (时间片), each user/process/job/task occupies one timeslot alternatively
 - ▣ **response time (响应时间)** should be < 1 second
 - ▣ each user has at least one program executing in memory $\Rightarrow$ **process**
 - ▣ if several jobs ready to run at the same time $\Rightarrow$ **CPU scheduling**
 - ▣ if processes don't fit in memory, **swapping (交换)** moves them in and out to run
 - ▣ **virtual memory (虚拟内存)** allows execution of processes not completely in memory



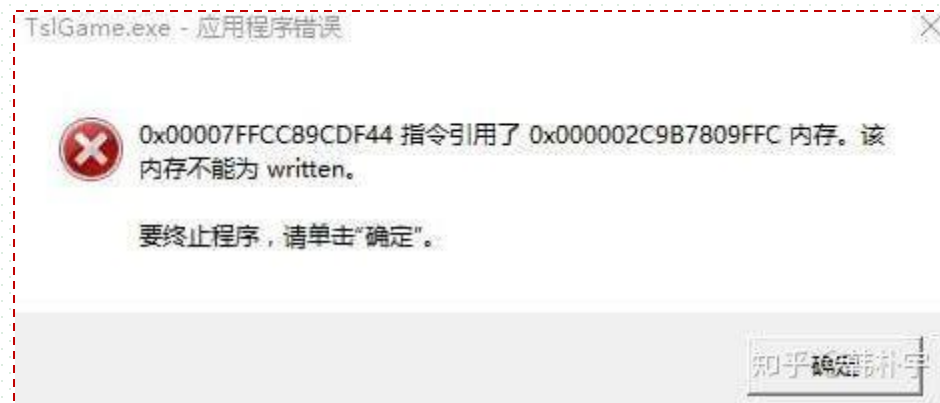
1990年图灵奖获得者 Fernando Jose Corby Corbato



- 费尔南多.何塞.科尔巴托 (1926-2019)
 - 计算机系统专家, MIT教授
 - 毕业于: 加州大学伯克利分校, 加州理工学院
- 学术贡献
 - 1961年, 领导研制了世界上第一个**分时系统CTSS** (Compatible Time-Sharing System, 名称中有Compatible这个词, 是因为它作为分时系统, 也能“兼容”批处理)
 - 开创了以**交互方式**由多用户同时共享计算机资源的新时代, 成为计算机发展历史上具有里程碑性质的重大突破
 - 分时系统的实现是计算机真正走向普及的开始
 - 1969年, 在其领导下, MIT、通用电气公司、贝尔实验室研制了著名的**通用操作系统MULTICS** (多路信息计算系统, Multiplexed Information and Computer System)
 - MULTICS有效地将计算机资源分配给多个远程用户程序, 同时解决了安全和保密等问题
 - 首次在大型系统软件中全面地使用结构化程序设计方法, 并用当时推出不久的高级程序设计语言PL/I编写了功能上独立于机器的全部系统程序

1.5 Operating-System Operations

- Important concepts
 - ▣ CPU Dual-mode operation (双模操作) & privilege instructions(权限/特权指令)
 - ▣ user mode/space vs kernel mode/space
- Interrupt driven (hardware and software)
 - ▣ hardware interrupt by one of the devices
 - ▣ software interrupt (**exception/异常** or **trap/陷阱**)
 - software error (e.g., division by zero)
 - request for operating system service, e.g. system call
 - other process problems include infinite loop, processes modifying each other or the operating system



Dual-mode operation & Privilege instructions

■ CPU privilege instruction

- ▣ identify machine instructions that may cause harm, and designate these instructions as privileged instructions (特权指令, 权限指令), i.e. *dangerous* instructions, e.g.
- ▣ e.g. privileged “set value of timers”
vs non-privileged “read the clock”
- ▣ e.g. 设置页表或者中断向量表的地址, 修改CPU工作模式
 - x86体系结构下, OS内核在系统初始化过程中设置中断向量表在内存中的基地址, 初始化内存中的中断向量表, 然后通过 权限指令LIDT将基地址设置到物理 CPU 的 IDTR (interrupt descriptor register) 中

■ Goal

- ▣ protect OS and other system components from errant users (i.e., programs), or errant users from one another

Dual-mode operation & Privilege instructions

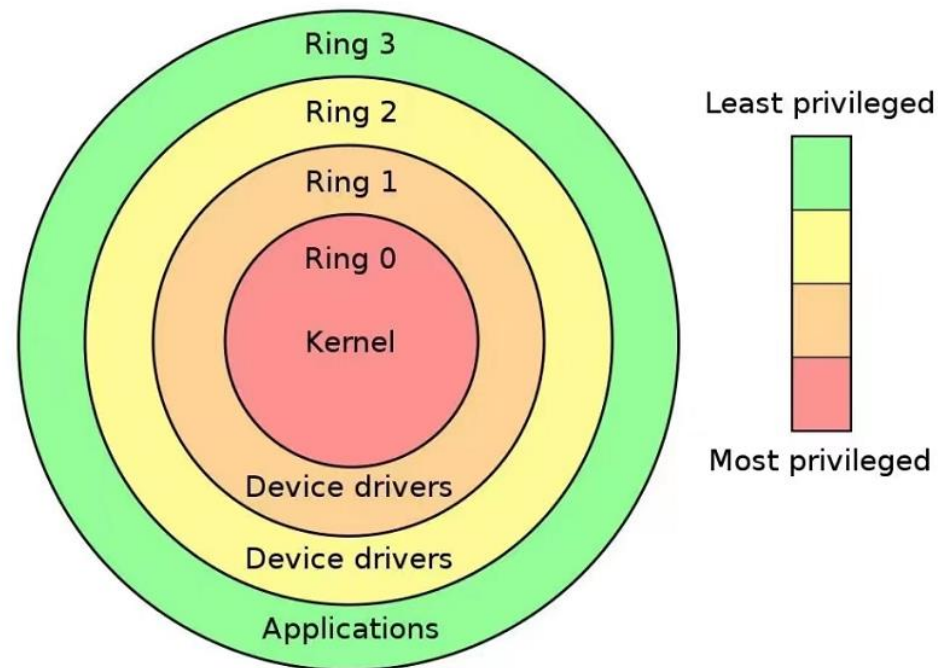
- Dual-mode: user mode and kernel mode
 - ▣ mode bit provided by hardware/CPU to indicate the current operation mode
 - 0: kernel/monitor^{监督}/supervisor/system/privileged mode, in which OS and privileged instructions are running on CPU
 - 1: user mode, in which user programs are executing on CPU
 - ▣ privileged instructions are only executable in kernel mode

Dual-mode operation & Privilege instructions

- Dual-mode scheme provides ability to distinguish when system is running user code or kernel code
- When a computer start up, its CPU runs in the kernel mode
- If user programs in user mode attempt to execute privileged instructions, the hardware does not execute these instructions, but rather treats them as illegal operations
 - ▣ a **trap** is generated and CPU control is passed to OS to handle this errors
 - /* 避免用户程序执行有可能导致错误的权限指令（危险指令）

Intel x86 CPU 特权等级及内核态、用户态

- Intel x86 CPU 架构对于所运行执行的指令，划分了4个不同特权等级
 - ▣ ring 0, ring 1, ring 2, ring 3, 称为保护环(protection ring)
- 从ring0到ring3, CPU特权级别依次降低, 即可执行的指令数目依次减少
 - ▣ ring 0可执行CPU全部指令, 其中15条指令只能在ring 0执行
 - ▣ ring 3不允许访问硬件资源, 如内存、I/O端口、某些机器指令
- 不同操作系统使用不同特权等级
 - ▣ Linux 使用ring 0和ring 3两个特权等级
 - 运行在ring 0的程序被称之为内核态(Kernel Space)程序, 如OS内核
 - 运行在ring 3等级的程序被称为用户态(User Space)程序, 如应用程序
 - ▣ Windows
 - ring 0: OS内核代码和驱动程序
 - ring 2: 某些需要特权的代码, 如需要I/O权限的用户程序和系统程序
 - ring 3: 非特权代码, 几乎所有应用程序均运行在ring 3



- Linux 操作系统中, 运行在内核空间/态、用户空间/态的程序可访问的地址空间不一样

ARM CPU 特权等级及内核态、用户态

- ARM CPU共有37个寄存器，被分为若干个组(BANK)，包括
 - ▣ 31个通用寄存器：包括程序计数器(PC指针)，均为32位的寄存器
 - ▣ 6个状态寄存器：用以标识CPU的工作状态及程序的运行状态，均为32位，只使用了一部分
- ARM CPU支持7种运行模式
 - ▣ usr (用户模式)
 - CPU正常程序执行模式
 - 大多数应用程序运行在该模式下，此时某些被保护的系统资源是不能被访问的
 - ▣ 非用户模式，也称为特权模式(Privileged Modes)，包括
 - fiq (快速中断模式)：用于高速数据传输或通道处理
 - irq (外部中断模式)：用于通用的中断处理
 - svc (管理模式)：操作系统使用的保护模式
 - abt (数据访问终止模式)：当数据或指令预取终止时进入该模式，用于虚拟存储及存储保护
 - sys (系统模式)：运行具有特权的操作系统任务
 - und (未定义指令中止模式)：当未定义的指令执行时进入该模式，可用于支持硬件协处理器的软件仿真

ARM CPU 特权等级及内核态、用户态

- 除去用户模式和系统模式以外的5种又称为异常模式(Exception Modes)
 - ▣ 用于处理中断或异常，以及需要访问受保护的系统资源等情况
- 运行模式可以通过软件改变，也可以通过外部中断或异常处理改变
- 在每一种运行模式下均有一组相应的寄存器与之对应，即在任意一种运行模式下，可访问的寄存器包括15个通用寄存器(R0 ~ R14)、一至二个状态寄存器(CSPR,SPSR)和程序计数器(PC)

查看原

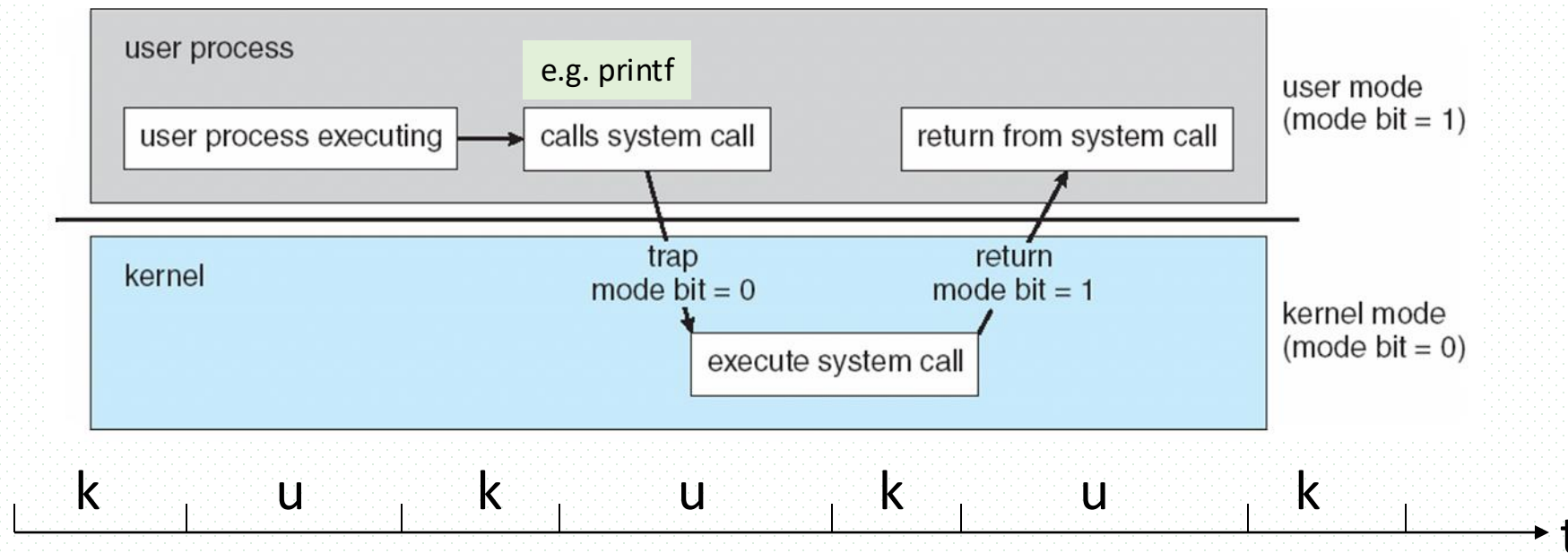
模式						
特权模式				异常模式		
用 户	系 统	管 理	中 止	未定义	中 断	快中断
R0	R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7	R7
R8	R8	R8	R8	R8	R8	R8_fiq
R9	R9	R9	R9	R9	R9	R9_fiq
R10	R10	R10	R10	R10	R10	R10_fiq
R11	R11	R11	R11	R11	R11	R11_fiq
R12	R12	R12	R12	R12	R12	R12_fiq
R13	R13	R13_svc	R13_abt	R13_und	R13_irq	R13_fiq
R14	R14	R14_svc	R14_abt	R14_und	R14_irq	R14_fiq
PC	PC	PC	PC	PC	PC	PC
CPSR	CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
		SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq

Address Space in Kernel/User Mode

- 运行在内核模式/内核态下的OS内核、运行在用户模式下的系统、应用程序所能访问内存地址空间(address space)是区分开的
 - 32-bit Windows, 4G address space,
 - 00000000~7FFFFFFF是用户内存区, 80000000~FFFFFFFF是内核内存区(2G用户, 2G内核)
 - 用户态代码不能访问内核态代码, 但内核态代码可以访问用户态代码
 - 32-bit Linux, 4G address space,
 - 内核态地址空间: 高端1G, 0xC0000000—0xFFFFFFFF
 - 用户态地址空间: 低端3G, 0x00000000— 0xBFFFFFFF
 - 64-bit Linux
 - 0000000000000000~00007FFEFFFEFFFF, 用户态地址空
 - FFFF800000000000~FFFFFFFFFFFFFFFF, 内核态地址空间
- 由OS提供的存储管理机制, 即页表page table + CPU MMU (Memory Management Unit) 控制内核态、用户态程序的内存地址访问访问

Transition from User to Kernel Mode

- At system boot time, CPU starts in kernel/monitor mode. After OS is loaded into memory and begin to work in normal states, CPU switches(切换) into user mode to run user programs
- System call changes mode to kernel, return from call resets it to user



Timer

- Timer to prevent infinite loop / process hogging resources
 - ▣ timer is set to interrupt the computer after some time period
 - ▣ timer implementation
 - keep a counter that is decremented by the physical clock.
 - operating system set the counter (privileged instruction)
 - when counter zero generate an interrupt
 - ▣ set up before scheduling process to regain control or terminate program that exceeds allotted time
 - e.g. in time-sharing systems, scheduling, CPU time slices are allocated to the processes to be scheduled, and the slices are controlled by timer

Kernel_timer in Linux

■ 头文件

■ linux/timer.h

■ 结构体(linux kernel 5.4.xx 以上)

```
struct timer_list {
    /*
     * All fields that change during normal runtime grouped to the
     * same cacheline
     */
    struct hlist_node    entry;
    unsigned long        expires;
    void                 (*function)(struct timer_list *);
    u32                  flags;

#ifdef CONFIG_LOCKDEP
    struct lockdep_map    lockdep_map;
#endif
};
```

■ 接口列表

```
#define timer_setup(timer, callback, flags) \
    __init_timer((timer), (callback), (flags))
extern void add_timer_on(struct timer_list *timer, int cpu);
extern int del_timer(struct timer_list * timer);
extern int mod_timer(struct timer_list *timer, unsigned long expires);
extern int mod_timer_pending(struct timer_list *timer, unsigned long expires);
extern int timer_reduce(struct timer_list *timer, unsigned long expires);
extern void add_timer(struct timer_list *timer);
extern int try_to_del_timer_sync(struct timer_list *timer);
extern int del_timer_sync(struct timer_list *timer);
```

■ 常用的接口

- timer_setup, 创建一个定时器
- mod_timer, 修改定时器超时时间, 在超时时重新设置
- del_timer_sync, 删除一个定时器 (多核)

例题

- 1. 下列选项中，操作系统提供给应用程序的接口是()
 - A.系统调用 B.中断 C. 库函数 D.原语

- 2. 下面的()资源不是操作系统应该管理的
 - A.CPU B.内存 C. 外存 D.源程序

- 3. CPU的状态分为用户态和内核态，从用户态转换到内核态的唯一途径是()
 - A.修改程序状态字指令 B.中断屏蔽 C. 中断 D.中断处理程序

- 4. 下列选项中，会导致用户进程从用户态切换到内核态的操作是是()
 - I.整数除以0 II. Sin()函数调用 III. Read系统调用
 - A. I、 II B.I、 III C. II、 III D.I、 II、 III

OS Management

- 1.6 Process management (进程管理)
- 1.7 Memory Management
- 1.8 Storage Management
- 1.9 Protection/保护 and Security/安全

1.6 Process Management

- A process is a program in execution. It is a unit of work within the system. Program is a **passive/被动 entity**, process is an **active/主动 entity**.
- Process needs resources to accomplish its task
 - ▣ CPU, memory, I/O, files
 - ▣ Initialization data
- Process termination requires reclaim of any reusable resources
- Single-threaded process has one **program counter (程序计数器)** specifying location of next instruction to execute
 - ▣ process executes instructions sequentially, one at a time, until completion
- Multi-threaded (多线程化) process has one program counter per thread
- Typically system has many processes, some user, some operating system running concurrently on one or more CPUs
 - ▣ concurrency by multiplexing (复用) the CPUs among the processes / threads (线程)

Process Management Activities

- The operating system is responsible for the following activities in connection with process management:
 - creating and deleting both user and system processes
 - suspending (挂起) and resuming (恢复) processes
 - providing mechanisms for process synchronization (同步)
 - providing mechanisms for process communication
 - providing mechanisms for deadlock handling (死锁处理)

1.7 Memory Management

- To execute a program all (or part) of the instructions must be in memory
- All (or part) of the data that is needed by the program must be in memory.
- Memory management determines what is in memory and when
 - ▣ optimizing CPU utilization and computer response to users
- Memory management activities
 - ▣ keeping track of which parts of memory are currently being used and by whom
 - ▣ deciding which processes (or parts thereof) and data to move into and out of memory
 - ▣ allocating and deallocating memory space as needed

1.8 Storage Management: File Systems

- OS provides uniform (一致, 均匀), logical view of information storage
 - ▣ abstracts physical properties to logical storage unit - **file**
 - ▣ each medium(存储介质) is controlled by device (i.e., disk drive, tape drive)
 - varying properties include access speed, capacity, data-transfer rate, access method (sequential or random)
- File-System management
 - ▣ files usually organized into directories
 - ▣ access control on most systems to determine who can access what
 - ▣ OS activities include
 - creating and deleting files and directories
 - Primitives (原语) to manipulate files and directories
 - mapping files onto secondary storage
 - Backup (备份) files onto stable (non-volatile) storage media

Mass-Storage (大规模/海量存储) Management

- Usually disks used to store data that does not fit in main memory or data that must be kept for a “long” period of time
- Proper management is of central importance
- Entire speed of computer operation hinges on disk subsystem and its algorithms
- OS activities
 - ▣ free-space management
 - ▣ storage allocation
 - ▣ disk scheduling
- Some storage need not be fast
 - ▣ tertiary storage includes optical storage, magnetic tape
 - ▣ still must be managed – by OS or applications
 - ▣ varies between WORM (write-once, read-many-times) and RW (read-write)

Caching: speeding up data access

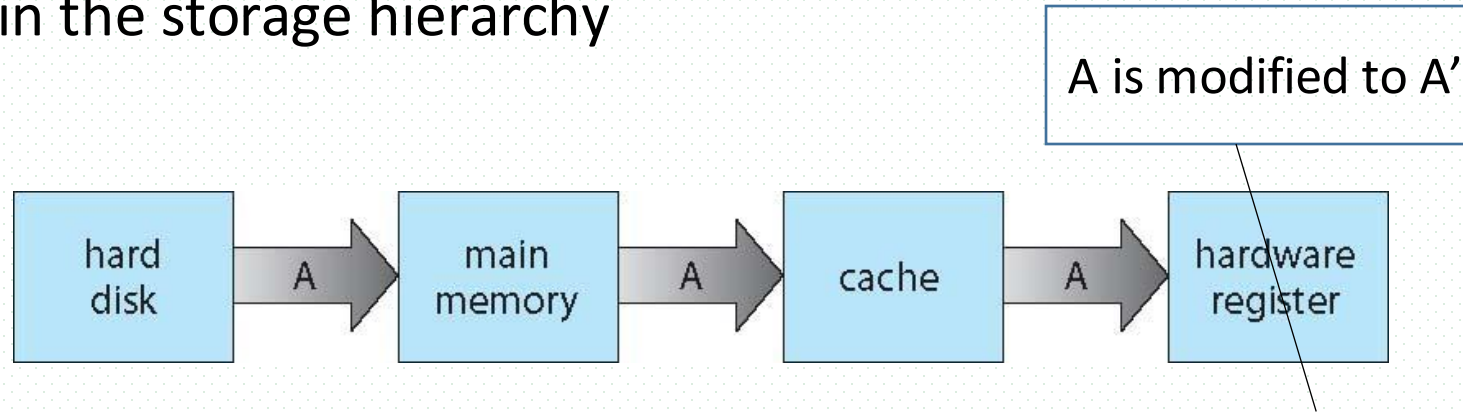
Performance of Various Levels of Storage

Level	1	2	3	4	5
Name	registers	cache	main memory	solid state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25 - 0.5	0.5 - 25	80 - 250	25,000 - 50,000	5,000,000
Bandwidth (MB/sec)	20,000 - 100,000	5,000 - 10,000	1,000 - 5,000	500	20 - 150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape

Movement between levels of storage hierarchy can be explicit or implicit

Migration of data “A” from Disk to Register

- Multitasking environments must be careful to use most recent value, no matter where it is stored in the storage hierarchy



- Multiprocessor environment must provide **cache coherency** in hardware such that all CPUs have the most recent value in their cache
- Distributed environment situation even more complex
 - ▣ Several copies of a datum can exist
 - ▣ Various solutions covered in Chapter 17

I/O Subsystem

- One purpose of OS is to hide peculiarities of hardware devices from the user
- I/O subsystem responsible for
 - ▣ memory management of I/O including buffering (storing data temporarily while it is being transferred), caching (storing parts of data in faster storage for performance), spooling (the overlapping (覆盖) of output of one job with input of other jobs)
 - ▣ general device-driver interface
 - ▣ drivers for specific hardware devices

1.9 Protection and Security

■ Protection

- ❑ any mechanism for controlling access of processes or users to resources defined by the OS

■ Security

- ❑ defense of the system against internal and external attacks
- ❑ Huge range, including denial-of-service, worms, viruses, identity theft, theft of service

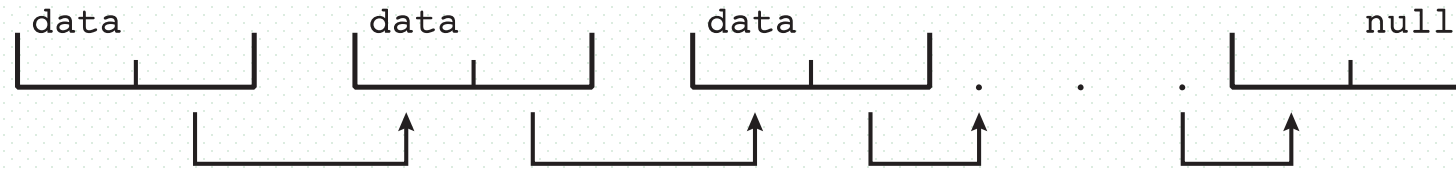
■ Systems generally first distinguish among users, to determine who can do what

- ❑ User identities (**user IDs**, security IDs) include name and associated number, one per user
- ❑ User ID then associated with all files, processes of that user to determine access control
- ❑ Group identifier (**group ID**) allows set of users to be defined and controls managed, then also associated with each process, file
- ❑ **Privilege escalation/权限升级** allows user to change to effective ID with more rights

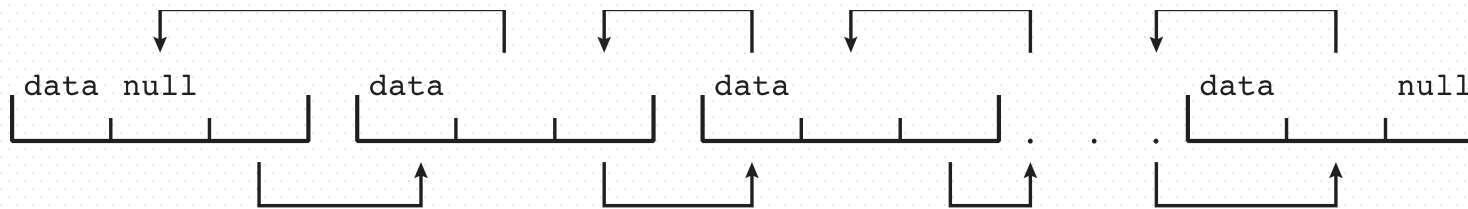
1.10 Kernel Data Structures 【略】

- Many similar to standard programming data structures

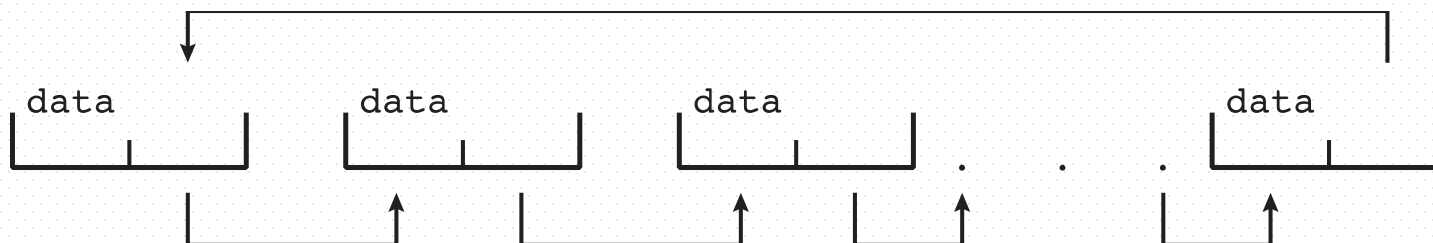
- **Singly linked list**



- **Doubly linked list**



- **Circular linked list**



- **Linked List Usage**

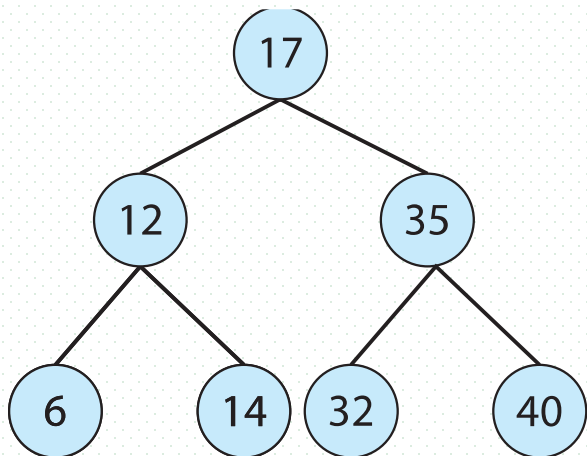
- ▣ in process scheduling queue, store the processes to be scheduled

Kernel Data Structures

■ Binary search tree

left $\leq$ right

- ❑ search performance is $O(n)$
- ❑ balanced binary search tree is $O(\lg n)$



■ 红黑树rbtree

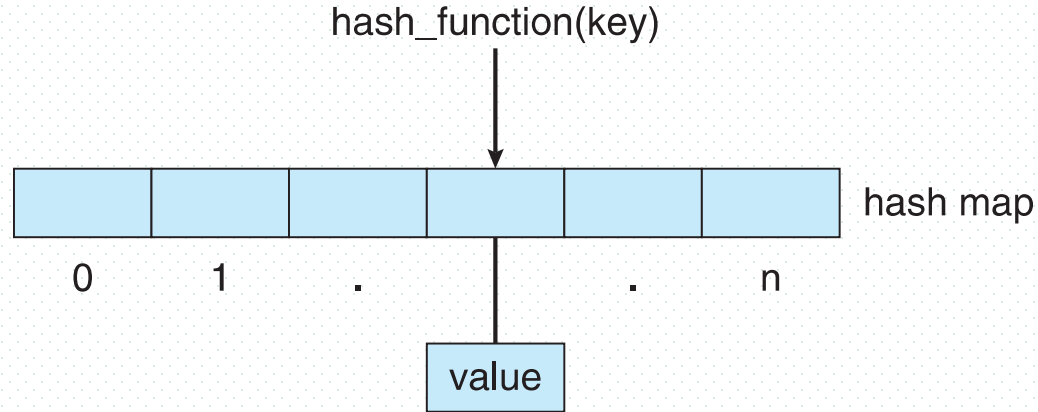
- ❑ rbtrees.h/rbtrees.c in Linux 3.0 kernel
- ❑ usage
 - CPU调度时, 组织running task(调度对象)
 - 高精度定时器timer, 组织定时请求
 - EXT3文件系统, 管理目录
 - 虚拟存储管理系统, 管理内存空间块VMAs (Virtual Memory Areas)

■ 基数树(radix-tree, 4.15版本+)

- ❑ process pid management

Kernel Data Structures

- **Hash function** can create a **hash map**



- **Bitmap/位图** – string of n binary digits representing the status of n items, used for
 - ▣ disk space management, process pid management(3.10版本)
- Linux data structures defined in
include files `<linux/list.h>`, `<linux/kfifo.h>`, `<linux/rbtree.h>`



Thanks for your
attention



北京邮电大学